

Sumário

1.	Construindo o core da aplicação:.....	2
1.1.	Ordem de desenvolvimento:	2
2.	Implementando LoginPage	2
2.1.	Implementando LoginPage.test.jsx	3
2.2.	Implementando LoginPage.jsx.....	4
3.	Implementando TarefasPage	5
3.1.	Implementando TarefasPage.test.jsx	5
3.2.	Implementando TarefasPage.jsx	6
3.3.	Deu erro, e agora?.....	7
3.3.1.	Refatorando o teste para operar de modo genérico.....	8
3.3.2.	Refatorando o teste para operar de modo determinístico	8
4.	Implementando RelatoriosPage:	9
4.1.	Implementando RelatoriosPage.test.jsx:	9
4.2.	Implementando RelatoriosPage.jsx:	10
4.2.1.	Funcionamento do RelatoriosPage.jsx	10
4.2.2.	Fluxo interno:	11
4.2.3.	Papel dentro da SPA:	11
5.	Implementação do globals.css	11
5.1.	globals.test.jsx	11
5.2.	globals.css	12
5.2.1.	Estrutura.....	13
5.2.2.	Funcionamento e papel do globals.css	13

1. Construindo o core da aplicação:

Com a estrutura da SPA devidamente refatorada e testada, incluindo a separação dos componentes Header, Footer e Layout, bem como a validação das rotas principais, a aplicação está pronta para receber seu núcleo funcional, ou seja, as páginas que compõem o fluxo real de uso da Pousada Pedra Furada.

Essa nova etapa marca a transição da estrutura estática para a camada lógica da aplicação, implementando os elementos interativos e as funcionalidades que simulam o comportamento do sistema real.

O desenvolvimento continua seguindo o ciclo TDD (Test-Driven Development) adotado até aqui: criar o teste / implementar o código / validar / refatorar.

1.1. Ordem de desenvolvimento:

A montagem do core da aplicação seguirá a seguinte lógica:

- LoginPage.test.jsx / LoginPage.jsx – Ponto de entrada da aplicação SPA.
- TarefasPage.test.jsx / TarefasPage.jsx – Página central com as tarefas diárias.
- RelatóriosPage.test.jsx / RelatóriosPage.jsx – Camada de exibição de relatórios
- globals.css – Validação e aplicação de estilo.

2. Implementando LoginPage

O componente `LoginPage.jsx` representa o ponto de entrada da aplicação, permitindo que o usuário informe suas credenciais para acessar as demais rotas da SPA.

Seu propósito é validar as credenciais fornecidas e conduzir o fluxo de navegação de forma controlada, garantindo uma interação intuitiva e segura.

O teste correspondente (`LoginPage.test.jsx`) tem a função de assegurar o comportamento correto do formulário de login, validando os dois cenários essenciais de uso:

- **Campos vazios → alerta de erro:** O sistema deve permitir um aviso impedindo o envio de dados incompletos.
- **Preenchimento correto → alerta de sucesso e redirecionamento:** O sistema deve exibir uma mensagem de boas-vindas personalizada e realiza o redirecionamento simulado para a página de tarefas (`/tarefas`) por meio do Hook `useNavigate`.

Durante os testes, o `MemoryRouter` do React é utilizado para simular o ambiente de navegação, permitindo testar o comportamento do `useNavigate` de maneira isolada e previsível, sem a necessidade de um navegador real.

A estrutura do teste segue o padrão adotado em toda a aplicação, com organização semântica e clareza descritiva.

Cada caso de teste é agrupado dentro de um bloco `describe("LoginPage")`, o que melhora a legibilidade dos relatórios do Jest e facilita a identificação dos cenários de validação.

Antes de cada execução, os mocks são redefinidos (`beforeEach(() => jest.clearAllMocks())`) para garantir que cada teste inicie em um estado limpo, evitando interferências entre execuções consecutivas.

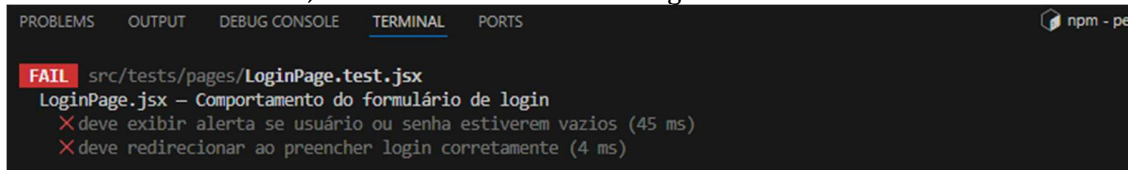
Esse conjunto de práticas assegura que o `LoginPage.test.jsx` ofereça uma cobertura completa dos principais comportamentos do login, mantendo consistência, clareza e confiabilidade dentro da suíte de testes da aplicação.

2.1. Implementando `LoginPage.test.jsx`

Adicione o código abaixo:

```
// src/tests/pages/LoginPage.test.jsx
import { render, screen, fireEvent } from "@testing-library/react";
import { MemoryRouter } from "react-router-dom";
import LoginPage from "../../pages/LoginPage";
describe("LoginPage.jsx – Comportamento do formulário de login", () => {
  beforeEach(() => {
    window.alert = jest.fn(); // mock do alert
    jest.clearAllMocks();      // limpa mocks antes de cada teste
  });
  test("deve exibir alerta se usuário ou senha estiverem vazios", () => {
    render(<MemoryRouter><LoginPage /></MemoryRouter>);
    // Simula clique no botão "Entrar" sem preencher campos
    fireEvent.click(screen.getByText("Entrar"));
    // Espera que apareça o alerta de campos vazios
    expect(window.alert).toHaveBeenCalled("Por favor, preencha usuário e
senha.");
  });
  test("deve redirecionar ao preencher login corretamente", () => {
    render(<MemoryRouter><LoginPage /></MemoryRouter>);
    // Preenche os campos
    fireEvent.change(screen.getByLabelText("Usuário:"), { target: { value: "Maria"
} });
    fireEvent.change(screen.getByLabelText("Senha:"), { target: { value: "123" }
});
    // Clica no botão "Entrar"
    fireEvent.click(screen.getByText("Entrar"));
    // Espera o alerta de sucesso
    expect(window.alert).toHaveBeenCalled("Bem-vindo, Maria!");
  });
});
npm test
```

Uma vez iniciado, o teste deve falhar com os seguintes erros:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS npm - pe
FAIL src/tests/pages/LoginPage.test.jsx
LoginPage.jsx - Comportamento do formulário de login
  ✗ deve exibir alerta se usuário ou senha estiverem vazios (45 ms)
  ✗ deve redirecionar ao preencher login corretamente (4 ms)
```

O teste não passou e o Jest está correto certo em reclamar, pois foi escrito um teste para um componente que ainda não existe.

2.2. Implementando LoginPage.jsx

Adicione o código abaixo:

```
// src/pages/LoginPage.jsx
import React, { useState } from "react";
import { useNavigate } from "react-router-dom";
export default function LoginPage() {
  const [usuario, setUsuario] = useState("");
  const [senha, setSenha] = useState("");
  const navigate = useNavigate();
  const handleLogin = (e) => {
    e.preventDefault();
    if (usuario && senha) {
      alert(`Bem-vindo, ${usuario}!`);
      navigate("/tarefas");
    } else {
      alert("Por favor, preencha usuário e senha.");
    }
  };
  return (
    <section>
      <h2>Login</h2>
      <form onSubmit={handleLogin}>
        <label>
          Usuário:
          <input
            type="text"
            value={usuario}
            onChange={(e) => setUsuario(e.target.value)}
          />
        </label>
        <br />
        <label>
          Senha:
          <input
            type="password"
            value={senha}
            onChange={(e) => setSenha(e.target.value)}
          />
        </label>
        <br />
        <button type="submit">Entrar</button>
      </form>
    </section>
  );
}
```

Em resumo, o `LoginPage.jsx` exibe o formulário de autenticação da Pousada Pedra Furada, controlando os dados digitados, validando o preenchimento e redirecionando o usuário para a página de tarefas após um login bem-sucedido.

3. Implementando TarefasPage

O componente `TarefasPage.jsx` representa a área principal de trabalho da aplicação, onde são exibidas e gerenciadas as tarefas diárias da pousada.

Seu papel é simular o painel de acompanhamento utilizado pela equipe, permitindo visualizar o status das tarefas e atualizá-las dinamicamente conforme o andamento das atividades.

Essa página tem importância central na SPA, pois representa a continuidade lógica do fluxo iniciado no login -> após a autenticação, o usuário é redirecionado para `/tarefas`, onde interage com dados reais ou simulados.

O comportamento desse componente é verificado por meio do teste `TarefasPage.test.jsx`, que assegura a atualização correta do status das tarefas, ao interagir com o botão “Atualizar”.

3.1. Implementando TarefasPage.test.jsx

O arquivo `TarefasPage.test.jsx` tem como objetivo garantir que a interface de tarefas responda corretamente às interações do usuário.

Esse teste cobre o cenário: verificar se o status da tarefa muda ao clicar no botão “Atualizar”.

Esse comportamento valida o uso correto do hook `useState` dentro do componente, confirmando que a lista de tarefas foi atualizada de forma reativa e exibida com o novo estado.

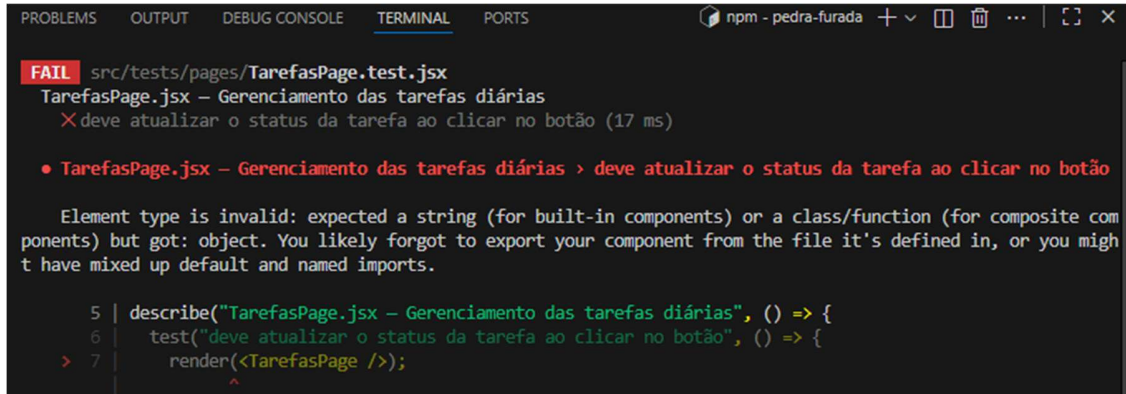
A estrutura do teste segue o mesmo padrão de clareza e semântica dos demais arquivos, utilizando `describe` para agrupamento e `fireEvent` para simular a interação do usuário.

Adicione o código abaixo:

```
// src/tests/pages/TarefasPage.test.jsx
import { render, screen, fireEvent } from "@testing-library/react";
import TarefasPage from "../../pages/TarefasPage";

describe("TarefasPage.jsx – Gerenciamento das tarefas diárias", () => {
  test("deve atualizar o status da tarefa ao clicar no botão", () => {
    render(<TarefasPage />);
    // Seleciona o primeiro botão "Atualizar"
    const botao = screen.getAllByText("Atualizar")[0];
    // Simula o clique do usuário
    fireEvent.click(botao);
    // Verifica se o status foi alterado para "Em andamento" ou "Concluída"
    expect(screen.getByText(/Em andamento|Concluída/)).toBeInTheDocument();
  });
});
```

Conforme esperado, o teste deve falhar pois ainda não foi construída a página TarefasPage.jsx



Proceda com o commit -m “test: Adição do teste TarefasPage.test.jsx e TarefasPage.jsx”

3.2. Implementando TarefasPage.jsx

O componente TarefasPage.jsx é responsável por exibir a lista de tarefas e permitir sua atualização dinâmica.

Ele utiliza o hook `useState` para armazenar o estado atual das tarefas e a função `atualizarStatus()` para modificar o status conforme o botão “Atualizar” é acionado.

Cada item da lista exibe o nome da tarefa e o seu status atual, sendo possível simular uma pequena progressão de trabalho (de “Atribuída” para “Em andamento” e, posteriormente, “Concluída”).

O componente TarefasPage.jsx é inteiramente controlado por estado e segue o padrão de renderização reativa do React.

A cada clique no botão “Atualizar”, o hook `useState` dispara uma re-renderização automática, refletindo na tela o novo status das tarefas.

Fluxo interno:

1. O componente inicia com uma lista fixa de tarefas (simulação de dados).
2. Cada item contém id, nome e status.
3. Ao clicar no botão “Atualizar”, a função `atualizarStatus()` identifica a tarefa clicada e modifica seu status sequencialmente.
4. A atualização do estado faz com que o React redesenhe a interface, exibindo o novo status.

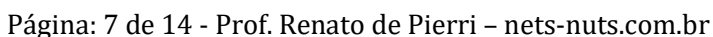
```
// src/pages/TarefasPage.jsx
import React, { useState } from "react";

export default function TarefasPage() {
  const [tarefas, setTarefas] = useState([
    { id: 1, nome: "Arrumar quarto 101", status: "Atribuída" },
    { id: 2, nome: "Lavar pátio", status: "Concluída" },
  ]);

  const atualizarStatus = (id) => {
    setTarefas((prev) =>
      prev.map((t) =>
        t.id === id
          ? {
              ...t,
              status:
                t.status === "Atribuída"
                  ? "Em andamento"
                  : t.status === "Em andamento"
                    ? "Concluída"
                    : "Concluída",
            }
          : t
        )
      )
    );
  };

  return (
    <section>
      <h2>Tarefas do Dia</h2>
      <ul>
        {tarefas.map((t) => (
          <li key={t.id}>
            {t.nome} - <strong>{t.status}</strong>{" "}
            <button onClick={() => atualizarStatus(t.id)}>Atualizar</button>
          </li>
        ))}
      </ul>
    </section>
  );
}
```

Uma vez implementado a página de tarefas, ela pode apresentar o erro:



O erro: `Found multiple elements with the text: /Em andamento|Concluída/` significa que o matcher da expressão regular (`/Em andamento|Concluída/`) encontrou mais de um elemento na tela que corresponde a esse padrão.

Analisando o código, no final da página temos duas correspondências válidas: `<strong>Em andamento</strong>` e `<strong>Concluída</strong>`, sendo que o `getByText()` aceita apenas um elemento.

3.3.1. Refatorando o teste para operar de modo genérico

Para corrigir de um modo genérico, pode-se utilizar no teste a função `getAllByText()` `expect(screen.getAllByText(/Em andamento|Concluída/).length).toBeGreaterThan(0);`

Essa abordagem verifica se existe pelo menos um elemento cujo texto seja “Em andamento” ou “Concluída”.

É um ajuste mais direto e alinhado com o propósito do teste original.

3.3.2. Refatorando o teste para operar de modo determinístico

Se optar para que ocorra a validação de cada um dos elementos esperados, é possível ajustar o teste para algo mais determinístico:

```
fireEvent.click(screen.getAllByText("Atualizar")[0]);
expect(screen.getByText("Em andamento")).toBeInTheDocument();
```

Utilizando essa abordagem garante que o primeiro item mudou de status, sem depender do status do segundo.

Utilize o código abaixo em `TarefasPage.test.jsx` para configurar o teste de maneira determinística:

```
// src/tests/pages/TarefasPage.test.jsx
import { render, screen, fireEvent } from "@testing-library/react";
import TarefasPage from "../../pages/TarefasPage";

describe("TarefasPage.jsx – Gerenciamento das tarefas diárias", () => {
  test("deve atualizar o status da tarefa ao clicar no botão", () => {
    // Renderiza o componente
    render(<TarefasPage />);

    // Seleciona o primeiro botão "Atualizar"
    const botao = screen.getAllByText("Atualizar")[0];

    // Simula o clique do usuário no primeiro item
    fireEvent.click(botao);
    // Verifica se o status do primeiro item mudou para "Em andamento"
    expect(screen.getByText("Em andamento")).toBeInTheDocument();
  });
});
```

O teste deve passar.

Commit -m: “refactor: Configurando TarefasPage.test.jsx para testar de maneira determinística”

4. Implementando RelatoriosPage:

O componente `RelatoriosPage.jsx` encerra o fluxo principal da SPA, sendo responsável por exibir as estatísticas gerais das tarefas da pousada.

O propósito é consolidar informações sobre o total de atividades realizadas, concluídas, em andamento e pendentes, funcionando como um painel de desempenho (dashboard).

Essa página tem caráter informativo e não interativo e o usuário apenas visualiza os dados gerados pelo sistema (ou simulados, nesta versão inicial).

O teste associado, `RelatoriosPage.test.jsx`, garante que os valores esperados sejam renderizados corretamente e que a estrutura da página mantenha a consistência visual e semântica.

4.1. Implementando RelatoriosPage.test.jsx:

O arquivo `RelatoriosPage.test.jsx` serve para validar se dados principais do relatório como: total de tarefas e número de tarefas concluídas são exibidos corretamente ao renderizar o componente.

Esse teste assegura que o componente carregue as informações de forma estática e previsível, refletindo corretamente os valores definidos no estado interno do relatório.

Adicione o código abaixo:

```
// src/tests/pages/RelatoriosPage.test.jsx
import { render, screen } from "@testing-library/react";
import RelatoriosPage from "../../pages/RelatoriosPage";

describe("RelatoriosPage.jsx – Exibição de relatórios de desempenho", () => {
  test("deve renderizar os dados do relatório corretamente", () => {
    render(<RelatoriosPage />);

    // Verifica os principais indicadores
    expect(screen.getByText("Total de tarefas: 10")).toBeInTheDocument();
    expect(screen.getByText("Concluídas: 6")).toBeInTheDocument();
    expect(screen.getByText("Em andamento: 3")).toBeInTheDocument();
    expect(screen.getByText("Pendentes: 1")).toBeInTheDocument();
  });
});
```

Esse teste é propositalmente simples, pois o componente ainda não possui lógica condicional nem interatividade.

Ele apenas garante que os elementos renderizados correspondam ao conteúdo esperado, um relatório estático com quatro indicadores principais.

Dado que ainda não foi implementada a página de relatório, o teste deve falhar:

```
FAIL src/tests/pages/RelatoriosPage.test.jsx
RelatoriosPage.jsx – Exibição de relatórios de desempenho
  ✕ deve renderizar os dados do relatório corretamente (16 ms)

  ● RelatoriosPage.jsx – Exibição de relatórios de desempenho > deve renderizar os dados do relatório corretamente

    Element type is invalid: expected a string (for built-in components) or a class/function (for composite components) but got: object. You likely forgot to export your component from the file it's defined in, or you might have mixed up default and named imports.

    5 | describe("RelatoriosPage.jsx – Exibição de relatórios de desempenho", () => {
    6 |   test("deve renderizar os dados do relatório corretamente", () => {
    7 |     render(<RelatoriosPage />);
      |           ^
    8 |   });
    9 | })
```

4.2. Implementando RelatoriosPage.jsx:

O componente RelatoriosPage.jsx é responsável por exibir o resumo numérico das tarefas.

Ele define um objeto de estatísticas, estatísticas, e o utiliza diretamente dentro do JSX, formatando os dados em uma lista estruturada.

Adicione o código abaixo:

```
// src/pages/RelatoriosPage.jsx
import React from "react";

export default function RelatoriosPage() {
  const estatisticas = {
    total: 10,
    concluidas: 6,
    emAndamento: 3,
    pendentes: 1,
  };

  return (
    <section>
      <h2>Relatórios de Desempenho</h2>
      <ul>
        <li>Total de tarefas: {estatisticas.total}</li>
        <li>Concluídas: {estatisticas.concluidas}</li>
        <li>Em andamento: {estatisticas.emAndamento}</li>
        <li>Pendentes: {estatisticas.pendentes}</li>
      </ul>
    </section>
  );
}
```

4.2.1. Funcionamento do RelatoriosPage.jsx:

O componente RelatoriosPage.jsx é puramente declarativo e estático, ele ainda não contém lógica de estado (useState) e nem de interação.

Seu papel é apresentar os indicadores de desempenho das tarefas de forma simples e acessível, permitindo ao usuário uma visão consolidada do progresso das atividades.

4.2.2. Fluxo interno:

1. O objeto `estatisticas` contém os valores numéricos do relatório.
2. Esses valores são injetados dinamicamente no JSX via interpolação (`{}`), garantindo clareza e legibilidade.
3. A estrutura de saída é composta por um `<section>` com título e uma `<ul>` listando cada indicador.
4. O teste valida a integridade dessa renderização e assegura que o relatório exibe corretamente os dados esperados.

4.2.3. Papel dentro da SPA:

O `RelatoriosPage.jsx` atua como a etapa final do fluxo de navegação da Pousada Pedra Furada, consolidando as informações operacionais exibidas nas demais telas (Login e Tarefas).

Ele pode futuramente ser expandido para incluir gráficos, filtros de período ou integração com APIs, mantendo a base semântica já testada.

5. Implementação do `globals.css`:

O arquivo `globals.css` é responsável por definir o estilo visual padrão da aplicação, estabelecendo uma base de aparência e comportamento para todos os componentes da SPA.

Ele garante que os elementos apresentem uma aparência uniforme e coerente, independentemente da página renderizada (Login, Tarefas ou Relatórios).

Diferente dos componentes JavaScript, o CSS não possui comportamento lógico a ser testado.

Ainda assim, é importante validar sua presença e integridade, assegurando que o arquivo seja corretamente importado e incluído no processo de build.

Essa verificação evita erros de importação ou falhas durante a compilação da aplicação com o Next.js ou React.

5.1. `globals.test.jsx`:

Como arquivos CSS não contêm lógica executável, o teste associado (`globals.test.jsx`) tem um propósito puramente de integração: garantir que o arquivo exista e seja importável sem erros.

Isso é útil para detectar problemas de caminho, falhas de empacotamento (como em builds Webpack/Vite) ou erros de importação que possam impedir o carregamento do estilo global durante a execução.

Adicione o código abaixo:

```
// src/tests/styles/globals.test.jsx
test("o arquivo de estilos globais deve ser importável", () => {
  const css = require("../styles/globals.css");
  expect(css).toBeDefined();
});
```

O teste utiliza `require()` em vez de `import`, pois não se trata de um módulo JavaScript convencional.

O método `expect(css).toBeDefined()` verifica se o arquivo é encontrado e importado com sucesso.

Se o arquivo for renomeado, movido ou excluído, o Jest emitirá erro de importação, sinalizando a falha de integração.

Esse tipo de teste é simples, rápido e preventivo, muito útil em projetos que passam por refatorações constantes de estrutura de pastas.

5.2. globals.css:

O arquivo `globals.css` define as regras de estilo aplicáveis a toda a aplicação, as propriedades foram configuradas para criar uma interface limpa, legível e agradável, com foco em legibilidade e conforto visual.

Adicione o código abaixo:

```
/* src/styles/globals.css */

/* Define a base visual da aplicação */
body {
  margin: 0;
  font-family: system-ui, Arial, sans-serif;
  background-color: #f4f6f8;
}

/* Define o layout principal das páginas */
main {
  max-width: 900px;
  margin: 1rem auto;
  background: white;
  border-radius: 8px;
  padding: 1rem;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

/* Define o espaçamento padrão entre botões */
button {
  margin-left: 0.5rem;
}
```

5.2.1. Estrutura:

- **body:** define o fundo, a tipografia e remove margens padrão.
- **main:** centraliza o conteúdo, define a largura máxima e adiciona bordas arredondadas e sombra.
- **button:** adiciona espaçamento lateral para melhor ergonomia visual.

5.2.2. Funcionamento e papel do globals.css:

O globals.css é carregado automaticamente no ponto de entrada da aplicação, geralmente importado em main.jsx ou _app.js, dependendo do framework.

Ele atua como uma folha de estilo global, garantindo consistência entre as páginas e evitando a repetição de regras básicas em cada componente.

Essa abordagem facilita futuras manutenções e amplia a escalabilidade do projeto: novas páginas ou componentes herdam automaticamente a mesma identidade visual sem necessidade de redefinir estilos.

6. Considerações e desenvolvimento futuro:

Com a finalização desta etapa, a SPA Pousada Pedra Furada atingiu um importante milestone: a construção completa do núcleo da aplicação, contemplando as três páginas principais: Login, Tarefas e Relatórios. Todas estruturadas e testadas de acordo com o ciclo TDD (Test-Driven Development).

A jornada começou pela refatoração da base estrutural, com a criação e modularização dos componentes Header, Footer e Layout, garantindo uma arquitetura limpa, organizada e de fácil manutenção.

Em seguida, o foco se voltou à implementação funcional das páginas, sempre acompanhadas de seus respectivos testes unitários, assegurando comportamento previsível e integridade dos fluxos de navegação.

- O LoginPage marca o ponto de entrada da aplicação, simulando o processo de autenticação do usuário e o redirecionamento controlado entre rotas.
- O TarefasPage trouxe interatividade e estado reativo, permitindo atualizar dinamicamente o status das tarefas.
- O RelatoriosPage consolida as informações do sistema, apresentando um painel estático.
- Por fim, o globals.css trata da identidade visual da SPA, garantindo consistência estética em todas as páginas e componentes.

Cada teste criado teve como propósito: validar comportamento antes da implementação, reforçando o uso disciplinado do TDD como ferramenta de confiabilidade e documentação viva do sistema.

6.1. Próximos passos

Com a estrutura e o comportamento interno validados, o projeto deve continuar sua evolução.

O próximo desafio consiste em implementar uma API, responsável por:

- Fornecer autenticação real de usuários;
- Persistir e atualizar tarefas em um banco de dados;
- Gerar relatórios dinâmicos com base em dados reais;
- Integrar toda a lógica de front-end com back-end.

Essa evolução transformará a SPA da Pousada Pedra Furada de um protótipo testado em um sistema interativo e conectado.